# oauth2-passkey

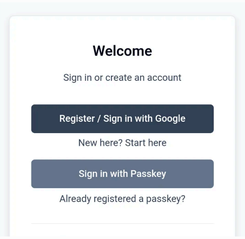
Passwordless Authentication Library for Rust

**Kimitoshi Takahashi** | Tokyo Rust Show & Tell | 2026/03/31

GitHub Author

## Try it Yourself!

passkey-demo.ccmp.jp




**Give it a try & tell me your thoughts!**

- *Admin access granted to all*
- *Privacy masked*
- *Ephemeral: data resets on restart*

## Motivation

- **OAuth2/OIDC**: delegate auth to trusted providers — no passwords to manage
- **Passkey**: phishing-resistant, biometrics/hardware key — no server-side secrets
- No integrated Rust/Axum library existed → built one → published to crates.io

## Flow: Google Login → Passkey Setup

**Step-by-step:**

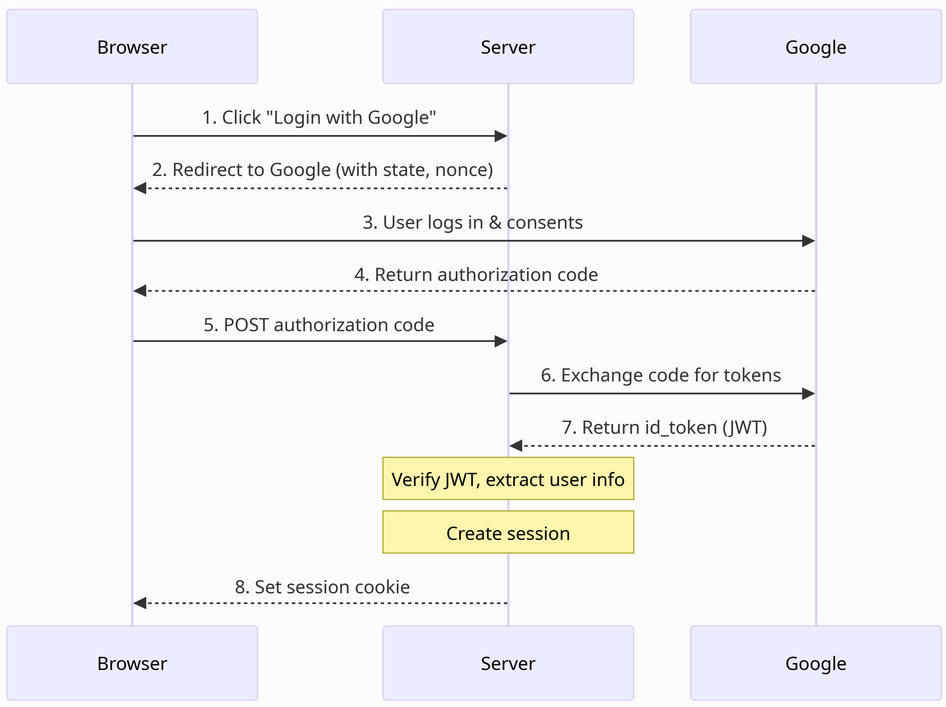
1. **OAuth2 Login**
   → create user linked with Google account
2. **Passkey Promotion**
   → register Passkey → link to user
3. **Next Login**
   → use either Passkey or Google OAuth2

**Internal DB structure:**

```
👤 users (id PK)
 │
 ├── 🌐 oauth2_accounts
 │     ├── Google ID
 │     └── GitHub ID
 │
 └── 🔑 passkey_credentials
       ├── Touch ID
       └── YubiKey
```

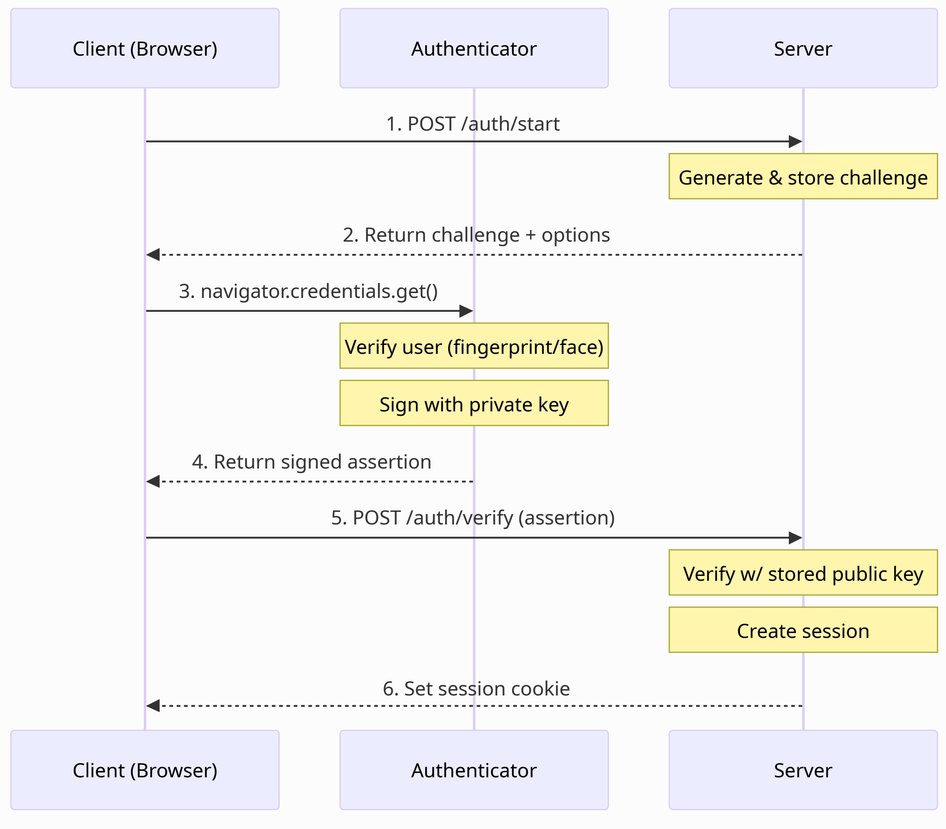
One user can have multiple OAuth2 accounts and multiple Passkey credentials.

## How OAuth2/OIDC Works




*Page-redirect: Google → code → id_token → session*

## How Passkey/WebAuthn Works




*JavaScript-driven: challenge → sign → verify → session*

## Usage: Add Auth in 3 Steps

**Step 1:** Configure `.env` (swap `sqlite` → `postgres`/`mysql` to change DB):

```
GENERIC_DATA_STORE_TYPE=sqlite
GENERIC_DATA_STORE_URL='sqlite:/tmp/auth.db'
GENERIC_CACHE_STORE_TYPE=memory
OAUTH2_GOOGLE_CLIENT_ID='xxx.apps.googleusercontent.com'
```

**Steps 2–3:** Initialize and merge router — login UI, account mgmt & admin panel built-in.

```rust
use oauth2_passkey_axum::{AuthUser, oauth2_passkey_full_router};  // 1. Import

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    dotenv().ok();
    oauth2_passkey_axum::init().await?;       // 2. Initialize
    let app = Router::new()
        .route("/", get(index))
        .merge(oauth2_passkey_full_router()); // 3. Merge router
    spawn_http_server(3001, app).await?;
    Ok(())
}
```

## Page Protection

**AuthUser extractor** (via `FromRequestParts`):

```rust
// Required auth — redirects to login if not authenticated
async fn protected(user: AuthUser) -> impl IntoResponse { ... }

// Optional auth — allows anonymous access
async fn public(user: Option<AuthUser>) -> impl IntoResponse { ... }
```

**Middleware** — choose 401 vs redirect, and skip DB when user info isn't needed:

```rust
let app = Router::new()
    .route("/api/1", get(h1)
        .route_layer(from_fn(is_authenticated_401)))
    .route("/api/2", get(h2)
      .route_layer(from_fn(is_authenticated_user_401)));

async fn h1(Extension(csrf): Extension<CsrfToken>)
{ ... }
async fn h2(Extension(user): Extension<AuthUser>)
{ ... }
```

**`is_authenticated_{Variant}`**

| Variant | Error | DB? | Ext. |
|---|---|---|---|
| 401 | 401 | No | CsrfToken |
| user_401 | 401 | Yes | AuthUser |
| redirect | Login | No | CsrfToken |
| user_redirect | Login | Yes | AuthUser |

## Storage & LazyLock Pattern

**Switch DB by changing `.env` only — no code changes:**

```
# Data store (pick one):
GENERIC_DATA_STORE_TYPE=sqlite
GENERIC_DATA_STORE_URL='sqlite:/tmp/auth.db'

# GENERIC_DATA_STORE_TYPE=postgres
# GENERIC_DATA_STORE_URL='postgres://id:pw@host:5432/db'

# GENERIC_DATA_STORE_TYPE=mysql
# GENERIC_DATA_STORE_URL='mysql://id:pw@host:3306/db'
```

```
# Cache store (pick one):
GENERIC_CACHE_STORE_TYPE=memory

# GENERIC_CACHE_STORE_TYPE=redis
# GENERIC_CACHE_STORE_URL=\
# 'redis://host:6379'
```

SQLite/PostgreSQL/MySQL abstracted via `DataStore` trait, held in a `LazyLock`:

```rust
static GENERIC_DATA_STORE: LazyLock<Mutex<Box<dyn DataStore>>>
    = LazyLock::new(|| { /* reads env, builds pool */ });
```

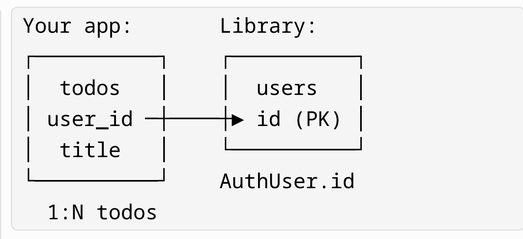
**Why LazyLock instead of Axum State?**

- **Axum State**: user composes `AppState`; 80+ internal fns all need `&State`
- **LazyLock**: just `init()` — globals accessed directly; fail-fast at startup
- **Trade-off**: one instance per process; use `#[serial]` in tests

## Integrating with Your App's Database

Your app adds its own tables linked by `AuthUser.id`:

```rust
async fn create_todo(
    State(state): State<AppState>,
    user: AuthUser, // from oauth2-passkey
    Form(form): Form<TodoForm>,
) -> Result<Response, ...> {
    db::create_todo(
      &state.pool,
      &user.id, // ──> todos.user_id
      &form.title
    ).await?;
    Ok(Redirect::to("/").into_response())
}
```




## Summary

- **Easy**: Add passwordless auth to your Axum app in minutes — Built-in UI included
- **Secure**: Passkey (phishing-resistant) + OAuth2, with CSRF protection and secure session cookies
- **Flexible**: Switch between SQLite, PostgreSQL, MySQL, and Redis with a single `.env` change